On Evaluating the Layout of UML Class Diagrams for Program Comprehension

Dabo Sun and Kenny Wong
Department of Computing Science
University of Alberta, Canada
{dabo,kenw}@cs.ualberta.ca

Abstract

UML class diagrams are helpful for understanding the structure of a software system. Algorithms and tools have been developed to generate UML class diagrams automatically for program understanding purposes. However, many tools often ignore perceptual factors in the layout of these diagrams. Therefore, users still have to spend much time and effort rearranging boxes and lines to make the diagram understandable. This paper presents key criteria and guidelines for the effective layout of UML class diagrams from the perspective of perceptual theories. Two UML tools have been analyzed and evaluated to illustrate how the criteria can be applied to improve the readability of class diagrams.

Keywords: *UML class diagrams, perceptual theory, aesthetics, graph layout, UML modeling tools.*

1 Introduction

The Unified Modeling Language (UML) [4] provides a graphical representation of a software system which can be used for both forward engineering and reverse engineering. UML class diagrams describe the classes and their various relationships such as generalization (inheritance), association (aggregation and composition), and other dependencies. A class diagram illustrates the static structure of a software system. Therefore, it is helpful for both modeling and understanding a system.

According to Tilley [30], UML diagrams, as a form of graphical documentation, can help software engineers to understand large-scale systems, but their efficacy depends on three main factors: syntax and semantics of UML, spatial layout of the diagrams, and domain knowledge. In particular, the spatial layout of UML diagrams plays a crucial role in fostering program understanding. UML tools have been developed to generate class diagrams automatically from source code. However, few tools can generate "nice" layouts for the diagrams [12].

Much research has been done for layouts of class diagrams. Early work explored graph drawing algorithms and aesthetics [7, 17]. Some new approaches have been proposed for graph layout specifically in the UML class diagram domain. Eiglsperger et al. proposed an algorithm based on the topology-shape-metrics approach for automatic layout of class diagrams, and it deals well with class diagrams with little or no structural information [13]. Eichelberger introduced a layout algorithm according to a large number of aesthetic criteria of UML class diagrams [11]. Gutwenger et al. introduced an approach for visualizing UML class diagrams conforming to a balanced mixture of aesthetic criteria [16]. Dwyer presented a three-dimensional UML class diagram representation using the Force Directed algorithm [10].

Purchase and her colleagues analyzed graph layout aesthetics in UML diagrams, focusing on user preferences, and conducted empirical studies of human comprehension to validate those aesthetic criteria and rank their effect [23, 24, 25, 27]. They also compared various UML notations and suggested which notations are more understandable [26].

However, since there are so many criteria and some of them conflict with each other, software engineers and tool designers are often overwhelmed and confused on choosing the appropriate criteria. Also, we observed that little research has been done on evaluating the layout functionality in existing UML modeling tools. This paper analyzes and classifies key criteria and guidelines for effective layout of UML class diagrams from the perspective of perceptual theories. We illustrate how the criteria can be applied by analyzing and evaluating the class diagram layout in two commercial tools: Borland Together [1] and Rational Rose [2].

The rest of this paper is organized as follows. Section 2 reviews cognitive theory to help understand the class diagram layout criteria. Section 3 introduces and classifies those criteria. Section 4 evaluates two UML tools, and

Proceedings of the 13th International Workshop on Program Comprehension (IWPC'05)
1092-8138/05 $ 20.00 IEEE

IEEE COMPUTER SOCIETY

section 5 summarizes this paper.

2 Perceptual theory

In this section, we outline several fields in cognitive science, such as the theory of perception, perceptual organization, and perceptual segregation. The laws of perception explain how our visual system identifies objects and how we put together the basic features to observe a coherent, organized world of things and surfaces. From the software visualization perspective, the principles of perceptual organization provide the basic design rules to organize multiple artifacts so that users can group related information and segregate useful information easily and without ambiguity. Perceptual theory can also be applied to evaluate the effectiveness of software visualization tools [29, 32]. In particular, the perceptual principles will help us to discover what kind of layout of UML diagrams would be more readable and understandable.

2.1 Theories of perception

It is widely accepted that visualization helps people to understand information, but how the brain processes, transforms, and interprets visual stimuli is still unclear. Various theories of perception have been proposed [22].

Marr's theory [20]: Cognitive functions are filters that operate on raw visual stimuli and turn them into information.

Gibson's theory [14]: We shift our attention based on the structure of the environment and create a cognitive map to interact with the world.

Gestalt theory [8, 21]: We restructure our perception in ways that make it unified and coherent. This theory formulates the principles of organization which explain why some displays are better than others. Perceptual organization will be discussed in more detail in the next subsection.

Theory of notation [22]: Many researchers try to find good notations for visualization which include symbol systems to create graphs that convey design semantics.

2.2 Perceptual organization

Perceptual organization refers to how objects in the world that we perceive are located and relate with one another [9]. Research in perceptual organization studies how small elements are grouped into larger objects [15], which is important for tool designers to devise an understandable visualization of a software system. The following are several important principles of perceptual organization, most of which are from the Gestalt Laws [9, 15].

Prägnanz (Good Figure): Prägnanz is a German word which means "suggestive figure". Therefore, the law of prägnanz is also called the law of *good figure* or the law of *simplicity*. That is, the images are perceived in such a way that their structures are as simple as possible. The figural "goodness" is sensitive to the amount of information necessary to describe a figure. Usually, simpler and more stable figure is considered a "good" figure [28].

Similarity: Similar elements (e.g., common shape or color) appear to be grouped together.

Continuation: Points tend to belong together if they result in straight or smoothly curved lines when connected, and lines are grouped together in such a way as to follow the smoothest path.

Proximity (Nearness): Elements that are close to each other are grouped together.

Familiarity (Meaningfulness): Elements are more likely to be grouped together if the groups seem familiar or meaningful.

Element Connectedness: Elements that are physically connected are perceived as a unit. In Figure 1, we perceive three dumbbells rather than some pairs of dots. It is interesting to notice that the dots that are next to each other in adjoint dumbbells are actually closer and according to the law of proximity, they should be grouped together. However, in this case, the law of connectedness overpowers the law of proximity.



Figure 1. Element connectedness

2.3 Perceptual segregation

Compared to perceptual organization, research in perceptual segregation basically studies the problem of "figure-ground segregation" to indicate when objects are separated [15]. Objects are usually referred to as figures and the background is called the ground. The fundamental figure-ground theory was proposed by Gestalt psychologists who believed that the figure is more like what we are familiar with and it is more memorable than the ground. They also stated that the figure is perceived as being in front of the ground, and the contour of the figure and the ground more likely belongs to the figure. Experiments show that the following factors make an object more like a figure:

Symmetry: Symmetric areas are usually seen as a figure.

Orientation: Horizontal or vertical orientations have higher probabilities to be seen as a figure than other orientations.

Contours: Modern theories about figure-ground segregation discovered that contours (i.e. edges) play a very important role in figure-ground perception [19].

IEEE COMPUTER SOCIETY

The perceptual theories unveil how humans perceive objects and interact with the environment, which may help us to understand why some kinds of UML diagram layouts are more readable than the others, so that we can better evaluate UML tool support.

3 UML class diagram layout criteria

According to Purchase et al. [24], not much research has been performed on the usability and understandability of graph drawing. Eichelberger also points out that the UML standard lacks a specification for the readability of the diagrams [12]. As a result, there are no common agreements on the criteria of readable and understandable layout of class diagrams. Our hypothesis is that criteria closely related to the laws of perception cannot be ignored, otherwise, our perceptual system might not be able to easily create correct "images" in our minds. Although semantics plays an important role in understanding UML diagrams, effective layout can substantially improve the quality of the diagram by increasing readability. In this section, we select key criteria from research results of graph drawing aesthetics [17, 23, 25, 33], various notation preferences for UML [26], UML layout aesthetics [11, 27], and UML style guidelines [5, 6] which can be directly related to perceptual principles, and we classify the criteria according to these principles. Figure 2 shows the classification of the selected criteria.

3.1 Law of good figure

C1: Join inheritance arcs. In [24], Purchase et al. state that inheritance arcs should be joined rather than separated lines. In Figure 3, (b) is preferable to (a), because notation (b) is more suggestive of a hierarchy than (a).

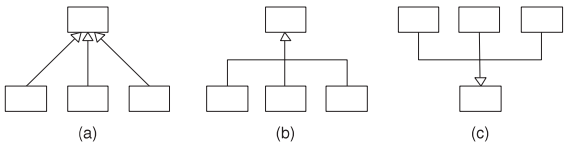


Figure 3. Notational variations of inheritance

C2: Represent association. The association name should be set beside the line, rather than in another association class linked to the line. In Figure 4, people prefer (a) to (b), because (b) has unnecessary complexity. Also, the association label should be set around the middle of the line [26].

C3: Be selective. Showing everything would not be helpful for understanding, especially for large systems.



Figure 4. Notational variations of association

The large volume and dense information makes diagrams difficult to read. Selective information is simpler and easier to group [6].

3.2 Law of similarity

C4: Use colors. Most people are sensitive to different colors. Applying colors to different groups of entities and relationships will help the user to distinguish things because objects in the same color are more similar and easier to be distinguished from objects in other colors [5].

3.3 Law of continuation

C5: Minimize edge crossings and bends. The number of edge crossings and bends should be minimized to make edges more continuous and easier to follow [33].

3.4 Law of proximity

C6: Center parents or children. A parent node and its child nodes should be placed as close as possible because they are closely related. This is supported by the law of proximity [11].

C7: Reduce length of edges. Edges should not be too long or too short because they make grouping and separation hard. This is supported by the law of proximity [27].

3.5 Law of familiarity

C8: Position superclasses above subclasses. A superclass should be above its subclasses and the inheritance arrows should be upwards. In Figure 3, (a) and (b) are preferable to (c) because people usually are familiar with putting superior objects on top of other objects [26].

3.6 Law of connectedness

C9: Avoid overlapping. Overlapping should be avoided, i.e., nodes should not overlap other nodes or edges. There might be two kinds of overlapping: edges overlapping nodes, and nodes overlapping nodes. We consider edges overlapping edges as *edge crossings* as described in C5. In Figure 5, (a) is confusing because it is

IEEE COMPUTER SOCIETY

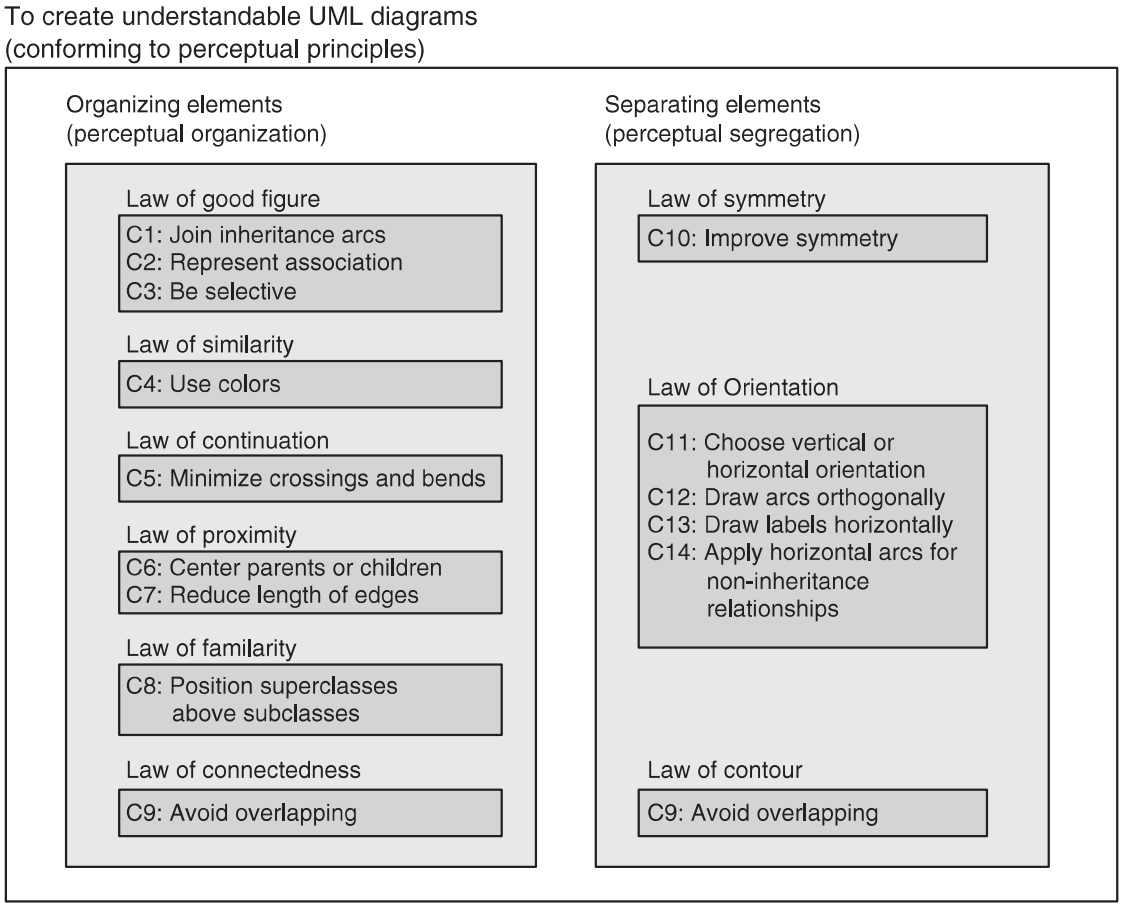


Figure 2. UML Class Diagram Criteria

not clear whether box A is connected to box B or directly connected to box C. As a result, it could be perceived as either (b) following the law of connectedness, or (c) following the law of continuation [11]. Probably most people perceive (a) as (b), as the law of connectedness overpowers he law of continuation in this case.

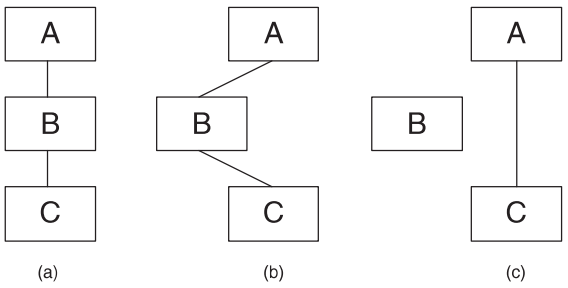


Figure 5. Effect of overlapping

3.7 Law of symmetry

C10: Improve symmetry. Symmetry of the diagram should be maximized, as symmetric areas are usually seen as a "good figure" [23].

3.8 Law of orientation

C11: Choose vertical or horizontal orientation. In general, most people prefer the orientation of left to right and top to bottom. Thus, the UML diagram should be drawn in vertical and horizonal directions [6].

C12: Draw arcs orthogonally. Nodes and edges should be arranged to an orthogonal grid, i.e., maximize the number of orthogonal edges [23].

C13: Draw labels horizontally. All the labels should be placed horizontally so that they can be read easily [24].

C14: Apply horizontal arcs for non-inheritance relationships. The common convention is to place relationships horizontally with the exception of inheritance [5].

IEEE COMPUTER SOCIETY